Encryption

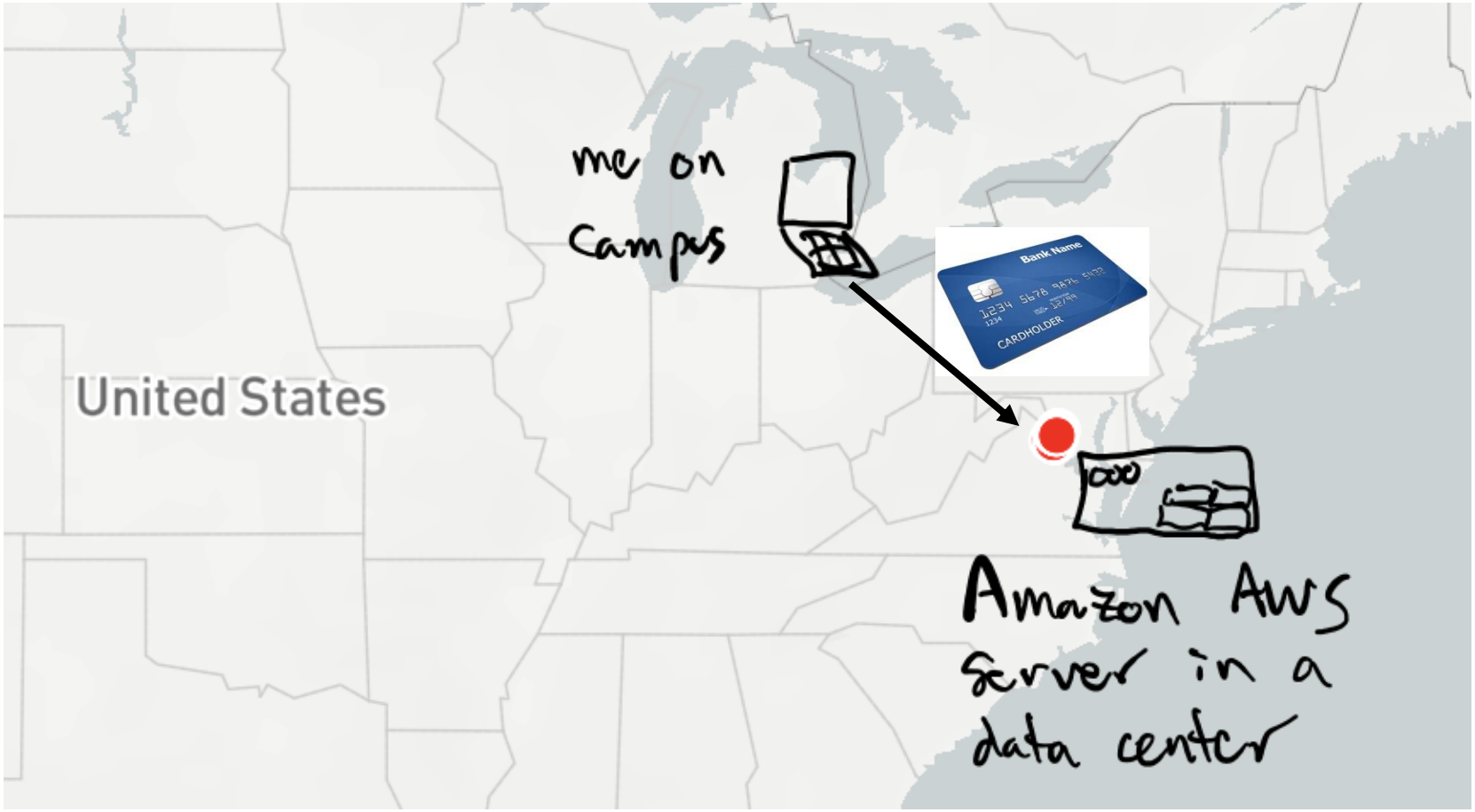
Agenda

- Overview
- Symmetric encryption
- Asymmetric encryption
  - Public key infrastructure
- Cryptographic hash functions
- Social impacts of encryption
- Summary

Network communication

- Two parties communicate over the internet
- Example: visiting `amazon.com`
- Let's imagine I'm sending my credit card info

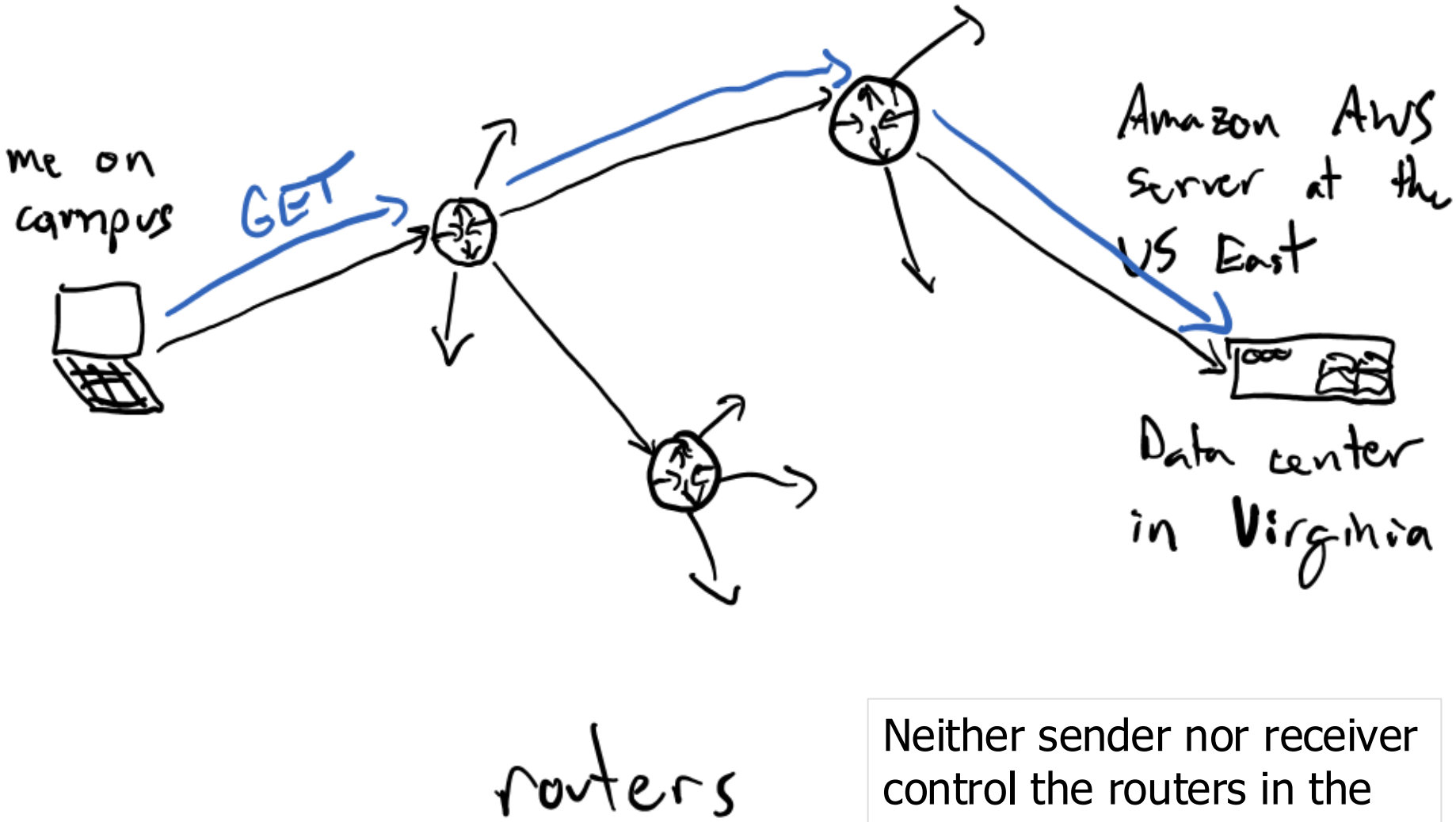
Network message



Internet routers

- How do we connect all the computers on the internet?
- A *router* forwards data from one network to the next
- Usually a purpose-built Linux computer with multiple network connections

Internet routers



Neither sender nor receiver control the routers in the middle!

Network security

- Two parties communicate over a network
- Assume powerful adversary
  - Can read (eavesdrop on) all data transmitted
  - Can modify or delete any data
  - Can inject new data

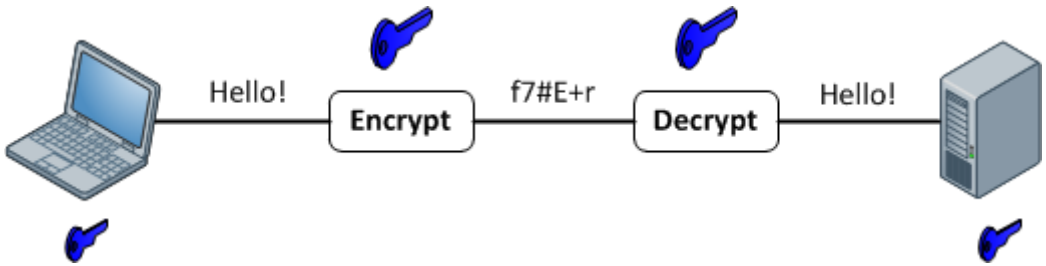
- Communication: What properties would you like?

Desirable properties

- Confidentiality
  - Adversary should not understand message
- Sender authenticity
  - Message is really from the purported sender
- Message integrity
  - Message not modified between send and receive
- Freshness
  - Message was sent “recently”
- Anonymity
  - Attacker should not know that we are communicating

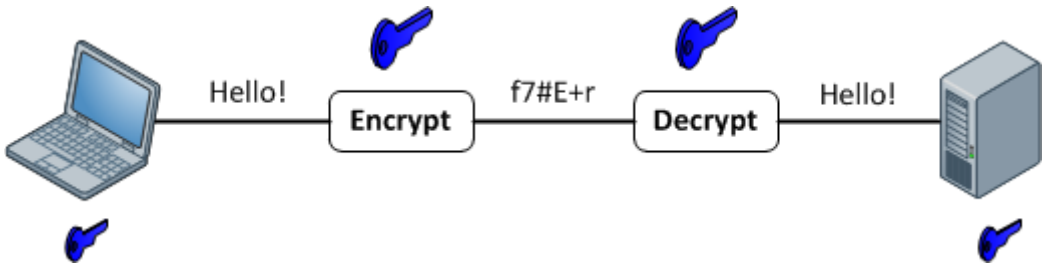
Encryption as a function

- Plaintext message string
- Encryption key (K_{enc})
- Decryption key (K_{dec})
- ciphertext = encrypt (plaintext, K_{enc})
- plaintext = decrypt (ciphertext, K_{dec})



Encryption in words

- Encryption applies a reversible mathematical function to data, producing unreadable output
- Decryption recovers the original data from the unreadable encryption-output
- The key is secret



Agenda

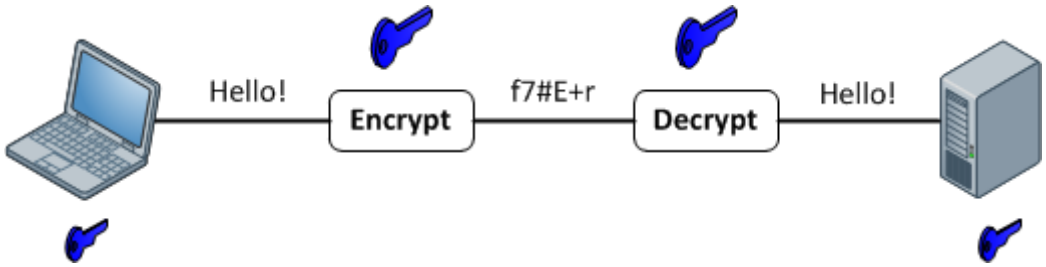
- Overview
- **Symmetric encryption**
- Asymmetric encryption
  - Public key infrastructure
- Cryptographic hash functions
- Summary

Symmetric encryption

- Key controls how data is encrypted and decrypted

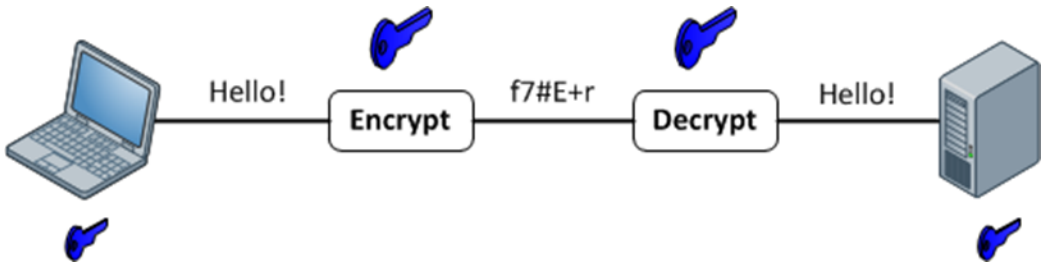
- Both sides use the same key
  - One side uses the key to encrypt
  - Other side uses the key to decrypt

- This is called *symmetric encryption*



Symmetric Encryption

- Advantage: fast to compute
- Disadvantage: how do you securely share the key?



AES

- "Advanced Encryption Standard"
- Most common symmetric cipher
- Won't go into AES details in this class

- Old symmetric cipher: DES
  - Now insecure

Symmetric encryption summary

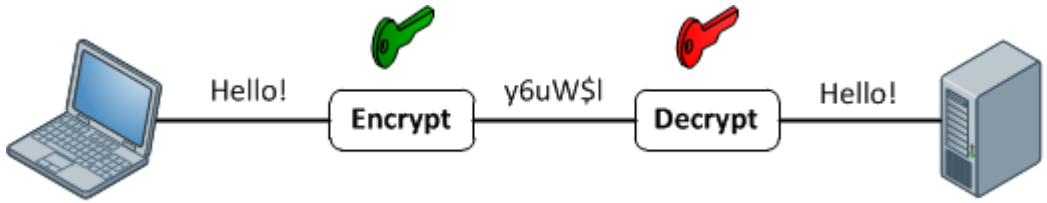
- One key for encrypt and decrypt
- Fast to compute
- Symmetric encryption provides *confidentiality*
  - Adversary should not understand message
- Symmetric encryption can provide *message integrity*
  - Message was not modified between send and receive

Agenda

- Overview
- Symmetric encryption
- **Asymmetric encryption**
  - Public key infrastructure
- Cryptographic hash functions
- Summary

Asymmetric encryption

- Asymmetric encryption uses a pair of keys
- Each party has:
  - A **public** key, which is published freely
  - A **private** key, which is shared with no one
- A message encrypted with one key can be decrypted with the other
- You can't derive one key from the other



Confidentiality

- Asymmetric encryption provides *confidentiality*
  - Adversary should not understand message
- Encrypt with public key, decrypt with private key

- Anyone can encrypt
  - ciphertext = encrypt (plaintext, K_{alice_public})
- Only Alice can decrypt
  - plaintext = decrypt (ciphertext, $K_{alice_private}$)

- Sometimes called ***sealing*** a message

Sender authenticity

- Asymmetric encryption provides *sender authenticity*
  - Message is really from the purported sender
- Encrypt with private key, decrypt with public key

- Only Alice can encrypt
  - ciphertext = encrypt (plaintext, $K_{alice_private}$)
- Anyone can decrypt
  - plaintext = decrypt (ciphertext, K_{alice_public})

- Sometimes called ***signing*** a message

Combining properties

- We can combine the two previous examples to gain all three desirable properties

- Confidentiality
  - Adversary should not understand message
- Sender authenticity
  - Message is really from the purported sender
- Message integrity
  - Message not modified between sending and receiving

Combining properties

Alice sends a message to Bob

1. sealed_ciphertext = encrypt(plaintext, K_bob_public)
2. signed_ciphertext = encrypt(sealed_ciphertext, K_alice_private)

Bob receives message from Alice

1. ciphertext = decrypt(signed_ciphertext, K_alice_public)
   - Verify signed ciphertext, anyone can do this
2. plaintext = decrypt(ciphertext, K_bob_private)
   - Decrypt ciphertext, only Bob can do this

How does asymmetric encryption work?

- Public key cryptography relies on trapdoor functions
  - A function that is:
    - Easy to compute in one direction
    - Hard to reverse without special information
  - “Easy” and “hard” are meant **computationally**

- Poor trapdoor functions
  - Add 2
  - Multiply by 3

- Finding good trapdoor functions in difficult

- The most widely used ones are based on **prime factorization**

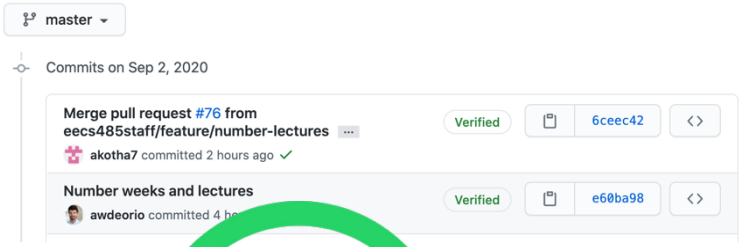
Trapdoor functions

- n = p*q, where p and q are large primes
  - Given p and q, easy to compute n
  - Given n, very hard to find p and q

- Public, private keys are derived from the original primes
- Only the product n is made public
- Recovering the original primes from n is computationally infeasible

Asymmetric encryption examples

- Encrypted email
  - Only the recipient can decrypt
- SSH keys
  - GitHub knows it's you when you git push
- Verified commits on GitHub
  - Real developer modified code
- Signal and WhatsApp



Signal

Agenda

- Overview
- Symmetric encryption
- Asymmetric encryption
  - **Public key infrastructure**
- Cryptographic hash functions
- Summary

How do you distribute public keys?

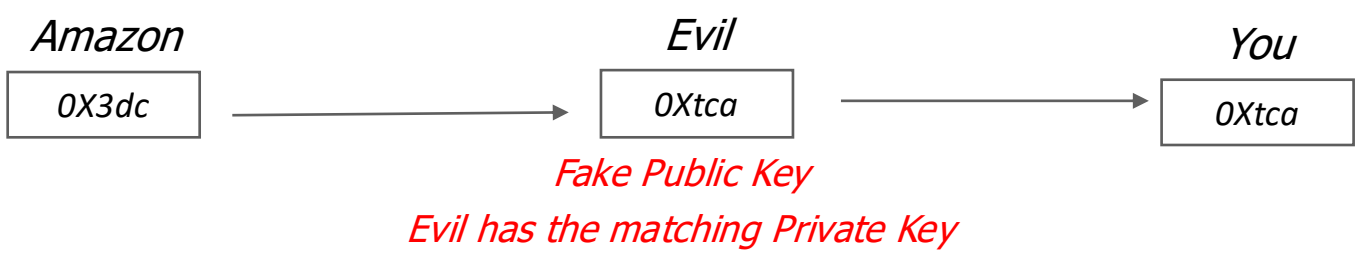
- Problem: How do you get a public key?
  - What if the public key is faked?
- Solution: The Public Key Infrastructure (PKI) distributes public keys safely, via certificates
- A *certificate* is signed document that contains org's public key

Public key infrastructure

- Asymmetric cryptography depends on sharing public keys
- How do you know that Amazon's public key actually comes from Amazon?

Public key infrastructure

- Asymmetric cryptography depends on sharing public keys
- How do you know that Amazon's public key actually comes from Amazon?



Certificate Authorities

- Verify identities and public keys.
- A *certificate* is signed document that contains org's name and public key
- Your browser has a copy of all certifications.

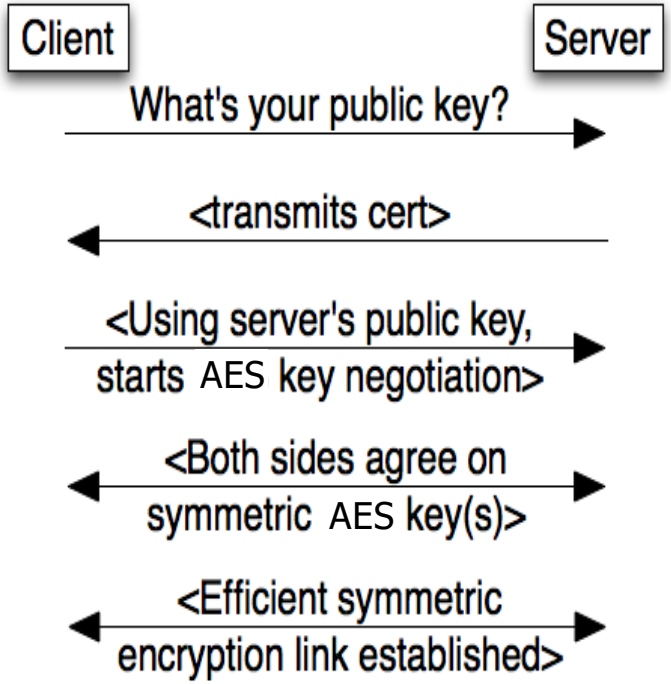
Asymmetric encryption is slow

- Problem: Asymmetric encryption is slow
- Symmetric is fast
- Solution:
  - Asymmetric to exchange symmetric key
  - Symmetric to encrypt all messages

- That's HTTPS (aka TLS/SSL)

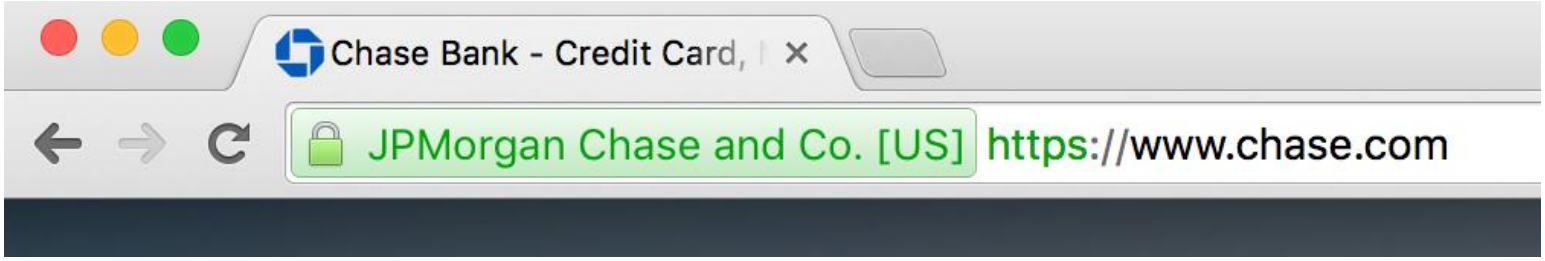
HTTPS

- Little public-key-encrypted data
- Browser verifies validity of certificate



HTTPS

- Transport Layer Security / Secure Sockets Layer
- Commonly, https://
- Encryption of all content that goes into TCP payload



HTTPS example

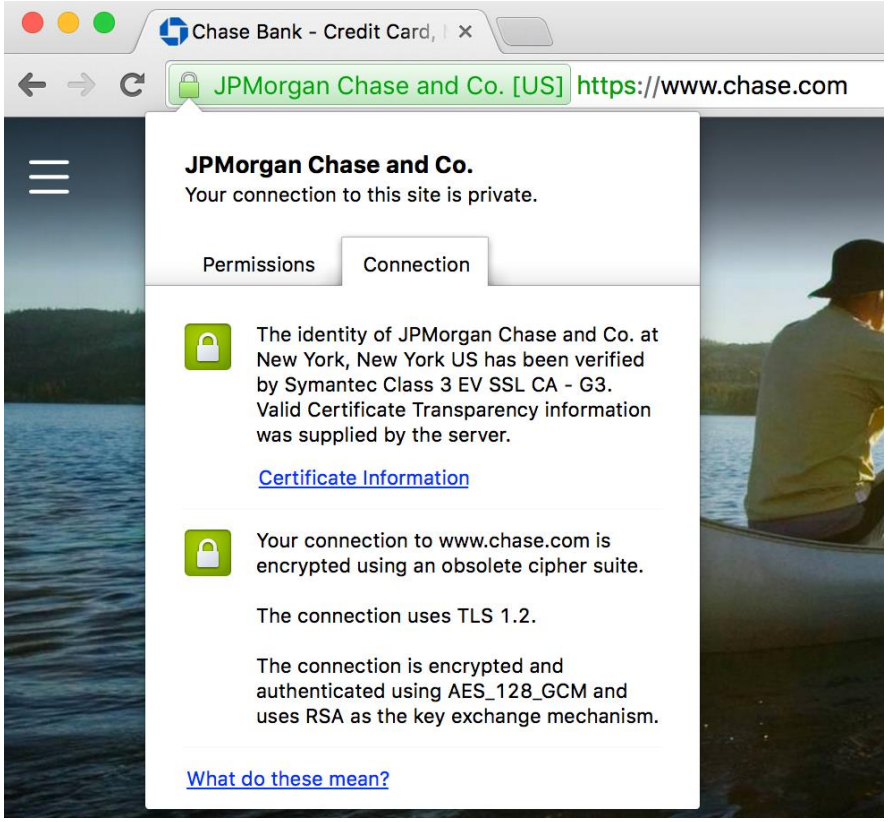
1. Hello

- Client sends hello message to server
  - Includes supported cipher algorithms and TLS version
- Server sends hello message to client
  - Includes selected cipher algorithm and TLS version

HTTPS example

2. **Certificate exchange**

- Server proves its identity to the client
- Server sends SSL certificate and public key
- Client checks certificate against stored CAs



HTTPS example

3. Key exchange

- Client generates random key to be used for later symmetric encryption

- Client encrypts this key using the server's public key
  - Remember, only the server will be able to decrypt this message using the server's private key

- After the shared key is established
  - All traffic is encrypted using fast symmetric encryption

Agenda

- Overview
- Symmetric encryption
- Asymmetric encryption
  - Public key infrastructure
- **Cryptographic hash functions**
- Summary

Hashing passwords

- How does the server verify your password?

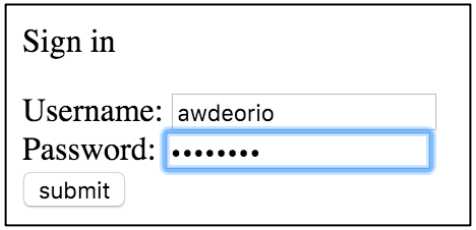
Hashing passwords

- How does the server verify your password?

- Server stores usernames and passwords in the database

Username	First Name	Last Name	Email	Password
Awdeorio	Andrew	DeOrio	awdeorio@umich.edu	Bob1pass
…….	…….	…….	…….	…….

- On login, compare the entered password with the stored one



- If the database is stolen; all passwords are exposed

Hashing passwords

- Better idea: store hashed passwords, not plain text
- Server hashes the password using a one-way hash function
- On login, hash the input password and compare hashes
- If the database is stolen, attackers do not get passwords directly

Username	First Name	Last Name	Email	Password
Awdeorio	Andrew	DeOrio	awdeorio@umich.edu	1xcrd4588x9…
…….	…….	…….	…….	…….

Example

- Using SHA-512 to hash a password

```
import hashlib
m = hashlib.sha512('bob1pass')
password_hash = m.hexdigest()
print(password_hash)
```

```
Af1bd47889bff89ccc889bc2aa61437c2ac90ee41161864
```

- You cannot recover the original password from the hash

Common passwords

- The most common passwords are:
  - 123456
  - 123456789
  - qwerty
  - password
  - 11111111
  - 12345678

- If an attacker gets my password database, how can they find which users used these passwords?

Cracking passwords

- **Brute force attack**: try every possible character combination, hash it, see if it matches db entry

- **Dictionary attack**: try all the words in the dictionary
  - Actually, many dictionaries

- Example: John the Ripper, Hashcat, etc.

Rainbow tables

- Pre-computed tables of hash
  - Example: download MD5 rainbow tables
  - Example: generate your own with RainbowCrack
- Compute (or download) once, reuse many times
- Can recover many passwords quickly

Hashed Password	Password
A347dsa4…….	Bob1pass
…….	…….

Protecting against cracking

- Rainbow tables assume that all the passwords were "hashed the same way"
- Use a ***salt*** to change how each password is hashed
- *Salt* is a random number added to password before hashing
- Each password is hashed with a different salt
- Store the salt with the password
- Now you would need a different rainbow table for every password!

Protecting against rainbow tables

Example:

hash(Salt + password)

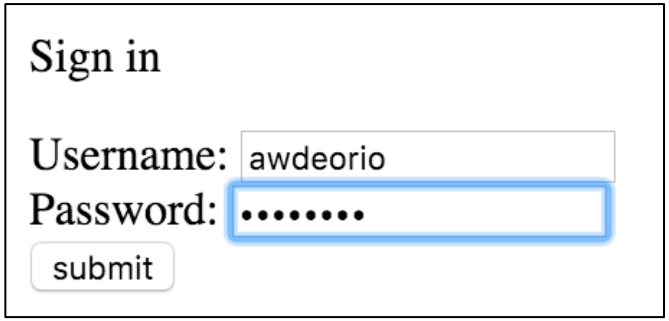
hash(3cmypassword) => 0x39f8e....

Username	First Name	Last Name	Email	Salt	Password
Awdeorio	Andrew	DeOrio	awdeorio@umich.edu	3c	0x39f8e…
…….	…….	…….	…….		…….

- Different for each password
- Store both salt and hashed passwords in DB.

Login verification

- User logs in

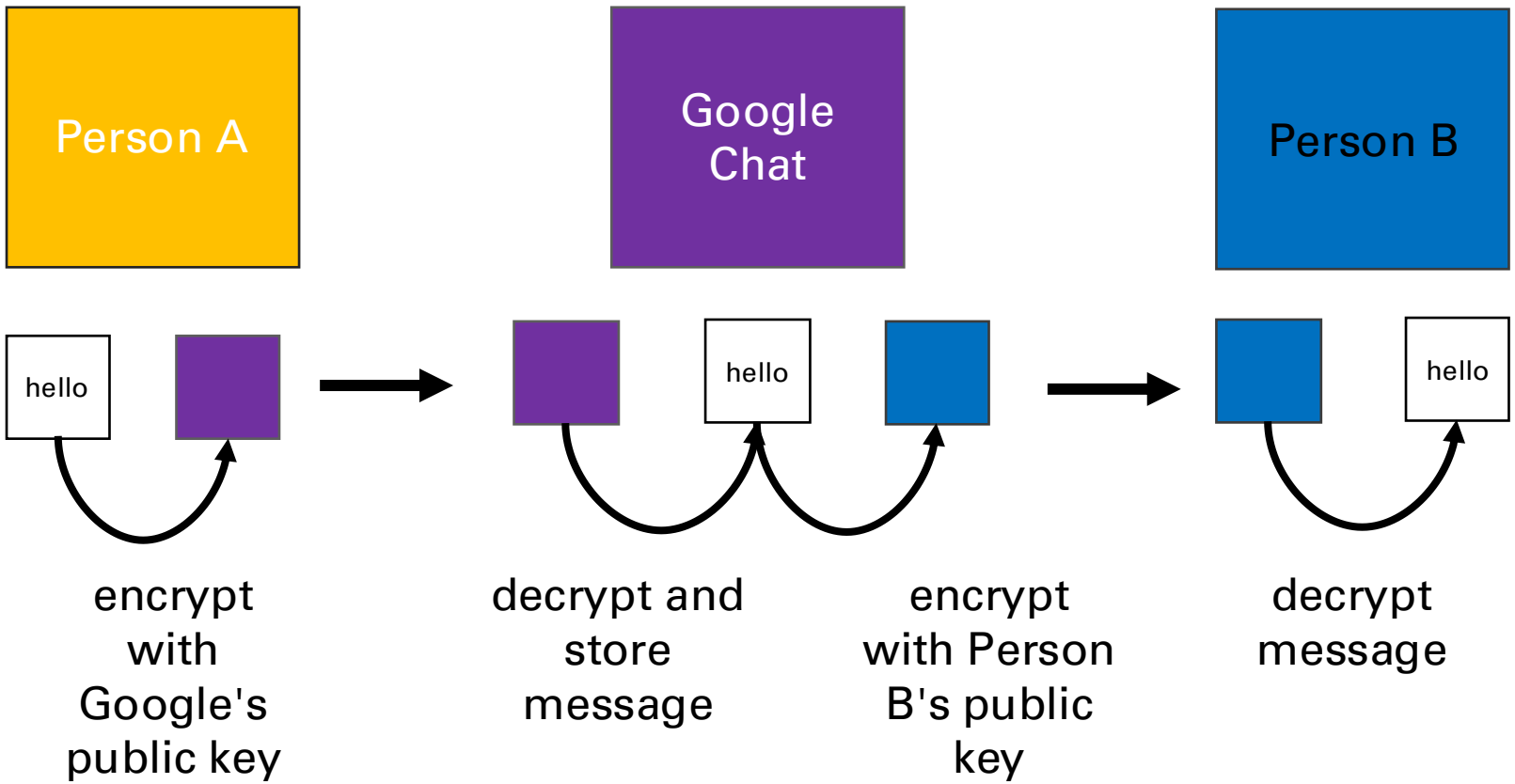


- Server reads password entry from database: `sha512$<SALT>$<HASHED_PASSWORD>`
- Server computes `sha512(<SALT> + input_password)`
- Server compares with stored hashed password `<HASHED_PASSWORD>`
- If equal; login succeeds

Agenda

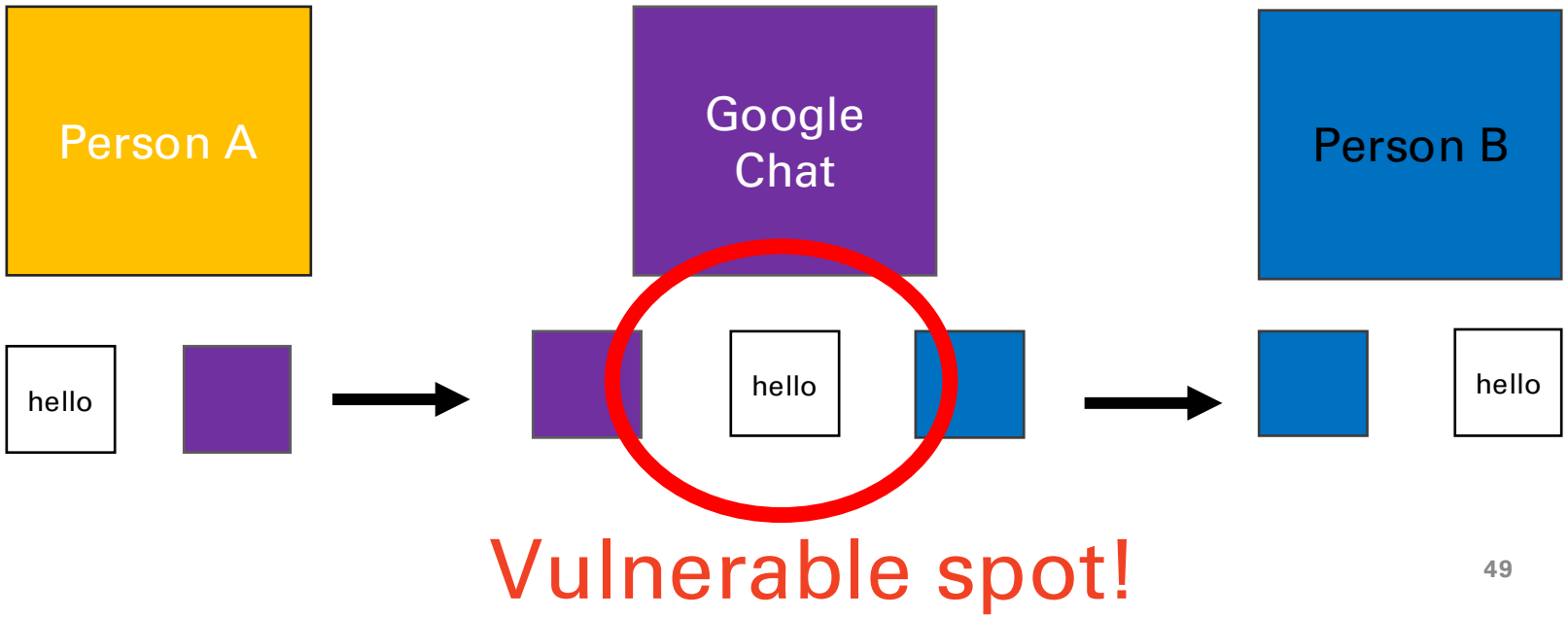
- Overview
- Symmetric encryption
- Asymmetric encryption
  - Public key infrastructure
- Cryptographic hash functions
- **Social impacts of encryption**
- Summary

Standard chat app

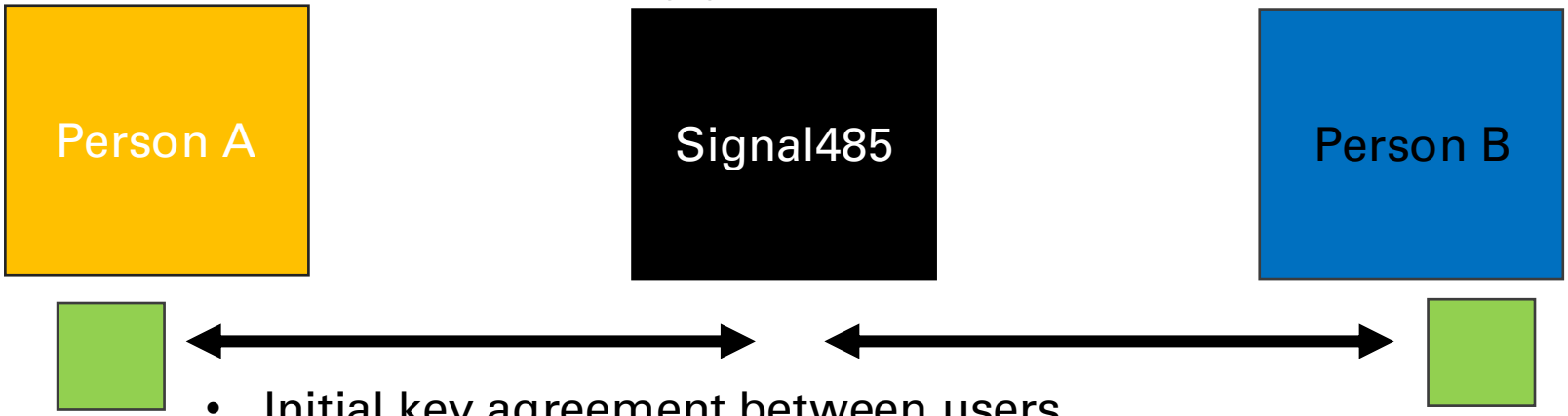


Issues

- Google can read all messages
- Law enforcement can subpoena messages without users knowing



End-to-end encrypted chat



- Initial key agreement between users
- Server only forwards encrypted messages
- Diffie-Hellman creates a shared key only users know

encrypt with key

server forwards message but can't decrypt it

decrypt message

End-to-end encrypted chat

- Why not just use public/private keys?
  - If a private key leaked, all past messages can be decrypted
  - Decryption with private key is proof that a given user was the recipient of the messages

- Real algorithm has some more safeguards

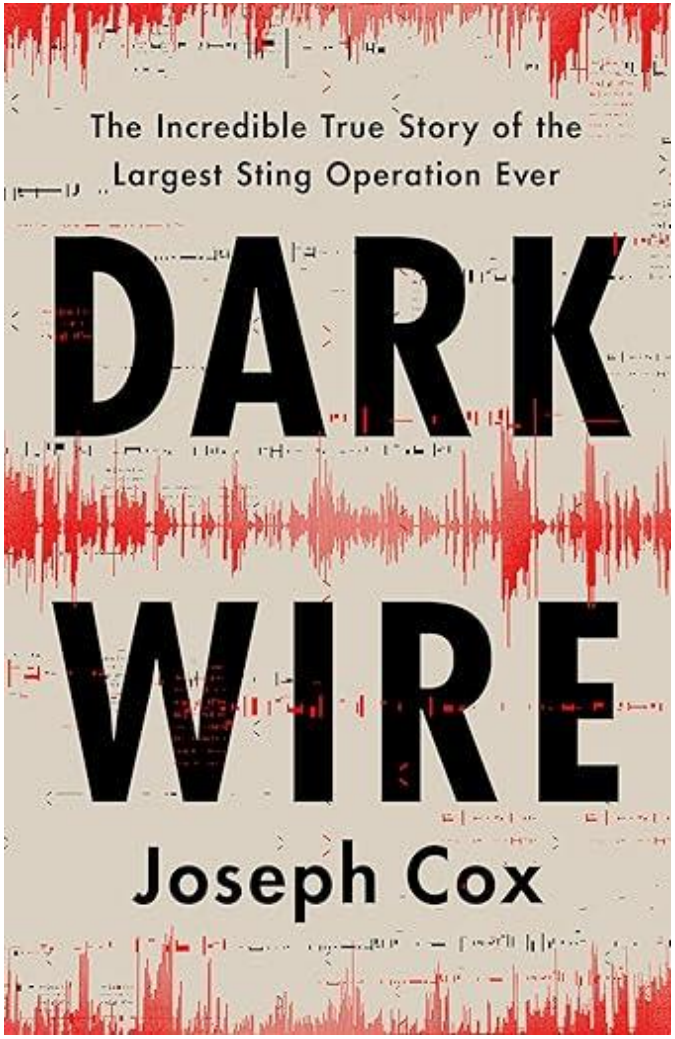
Policy response

- FBI: calls it "warrant-proof encryption"
  - "warrant-proof encryption has created lawless digital spaces where bad actors are
    - sexually exploiting children,
    - conducting human trafficking,
    - sharing terrorist propaganda,
    - …" (https://www.fbi.gov/how-we-investigate/lawful-access)

- Society must choose:
  - Strong privacy
  - Or easier law enforcement access

An interesting read

- Dark Wire by Joseph Cox
  - Non-fiction
- Anom end-to-end encrypted phone network
- FBI backdoor added by developer

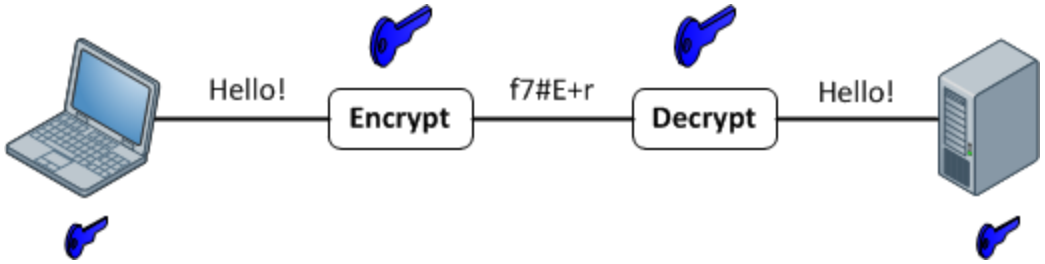


Agenda

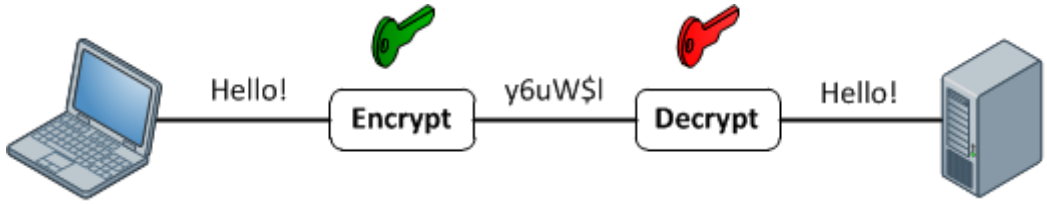
- Overview
- Symmetric encryption
- Asymmetric encryption
  - Public key infrastructure
- Cryptographic hash functions
- Social impacts of encryption
- **Summary**

Summary

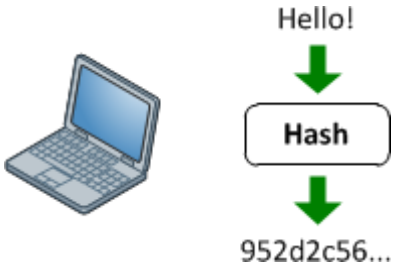
- Symmetric encryption
  - One key



- Asymmetric encryption
  - Two keys



- Cryptographic hash functions
  - No keys



Summary: desirable properties

<u>Today</u>

- Confidentiality
  - Adversary should not understand message
- Sender authenticity
  - Message is really from the purported sender
- Message integrity
  - Message not modified between send and receive

Summary: desirable properties

Next time

- Freshness
  - Message was sent “recently”

Later

- Anonymity
  - Attacker should not know that we are communicating
- Dark Web Lecture